

Intelligent Test Plan Agent - JIRA Dashboard Structure

Project Overview

Project Name: Intelligent Test Plan Agent

Project Code: AI_AGENT_JIRA

Objective: Design a web-based UI application that integrates with JIRA and LLM (Groq/Ollama) to automatically generate test plans from JIRA tickets.

EPIC 1: JIRA Integration Module

Epic Summary: Establish secure connection and data retrieval from JIRA

Story 1.1: JIRA Connection Setup

As a user, I want to configure JIRA API connection, **so that** I can fetch JIRA ticket details.

Acceptance Criteria:

- ☐ UI form to input JIRA API key
- ☐ UI form to input JIRA base URL
- ☐ Test connection button with validation
- ☐ Store credentials securely (local storage/env)
- ☐ Display connection status (connected/disconnected)

Tasks:

- TASK-1.1.1: Create JIRA connection settings UI
- TASK-1.1.2: Implement API key input field with mask
- TASK-1.1.3: Implement JIRA base URL input
- TASK-1.1.4: Create test connection functionality
- TASK-1.1.5: Store connection credentials securely

Story 1.2: Fetch JIRA Ticket Details

As a user, I want to enter a JIRA ID and fetch its details, **so that** I can use the ticket information for test plan generation.

Acceptance Criteria:

- ☐ Input field for JIRA ID (e.g., VW01)
- ☐ Fetch ticket summary, description, acceptance criteria
- ☐ Display fetched details in UI
- ☐ Error handling for invalid JIRA IDs
- ☐ Loading state during fetch

Tasks:

- TASK-1.2.1: Create JIRA ID input component
 - TASK-1.2.2: Implement JIRA API call to fetch ticket
 - TASK-1.2.3: Parse and display ticket details
 - TASK-1.2.4: Add error handling and validation
 - TASK-1.2.5: Add loading indicators
-

EPIC 2: LLM Integration Module

Epic Summary: Integrate with Groq.com and Ollama for AI-powered test plan generation

Story 2.1: Groq.com LLM Integration

As a user, I want to configure Groq.com API key, **so that** I can use cloud-based LLM for test plan generation.

Acceptance Criteria:

- ☐ UI to input Groq API key
- ☐ Store API key securely
- ☐ Test LLM connection
- ☐ Support multiple Groq models
- ☐ Display LLM status

Tasks:

- TASK-2.1.1: Create Groq settings UI
- TASK-2.1.2: Implement Groq API key input
- TASK-2.1.3: Create Groq client integration
- TASK-2.1.4: Add model selection dropdown
- TASK-2.1.5: Test LLM connectivity

Story 2.2: Ollama Local LLM Integration

As a user, I want to configure Ollama settings, **so that** I can use local LLM for offline test plan generation.

Acceptance Criteria:

- ☐ UI to enable/disable Ollama
- ☐ Input for Ollama base URL (default: localhost:11434)
- ☐ Model selection from available Ollama models
- ☐ Test Ollama connection
- ☐ Fallback mechanism if Ollama unavailable

Tasks:

- TASK-2.2.1: Create Ollama settings UI
- TASK-2.2.2: Implement Ollama enable/disable toggle
- TASK-2.2.3: Create Ollama client integration
- TASK-2.2.4: Fetch and display available models
- TASK-2.2.5: Test Ollama connectivity

Story 2.3: LLM Context Management

As a user, I want to add JIRA context to the LLM, **so that** the generated test plan is relevant to the ticket.

Acceptance Criteria:

- ☐ Format JIRA data for LLM prompt
- ☐ Include ticket summary, description, acceptance criteria
- ☐ Send context to selected LLM (Groq/Ollama)
- ☐ Handle LLM response
- ☐ Error handling for LLM failures

Tasks:

- TASK-2.3.1: Create context formatter
 - TASK-2.3.2: Build LLM prompt template
 - TASK-2.3.3: Implement context injection
 - TASK-2.3.4: Handle LLM responses
 - TASK-2.3.5: Add retry logic for failures
-

EPIC 3: Test Plan Generation Module

Epic Summary: Generate comprehensive test plans using JIRA data and templates

Story 3.1: Test Plan Template Management

As a user, I want to use predefined test plan templates, **so that** generated test plans follow a consistent structure.

Acceptance Criteria:

- ☐ Load testplan.pdf template from parent folder
- ☐ Parse template structure
- ☐ Store template sections (Test Objective, Scope, Test Cases, etc.)
- ☐ Allow template customization
- ☐ Support multiple templates

Tasks:

- TASK-3.1.1: Create template loader
- TASK-3.1.2: Parse PDF template structure
- TASK-3.1.3: Extract template sections
- TASK-3.1.4: Create template storage
- TASK-3.1.5: Build template selector UI

Story 3.2: AI-Powered Test Plan Generation

As a user, I want to generate a test plan from JIRA ID using AI, **so that** I can quickly create comprehensive test documentation.

Acceptance Criteria:

- ☐ Combine JIRA context with template
- ☐ Send to LLM for generation
- ☐ Parse LLM output into structured test plan
- ☐ Display generated test plan in UI
- ☐ Allow editing of generated content

Tasks:

- TASK-3.2.1: Create test plan generation service
- TASK-3.2.2: Build prompt combining JIRA + template
- TASK-3.2.3: Parse LLM response into sections
- TASK-3.2.4: Create test plan display UI
- TASK-3.2.5: Add inline editing capability

Story 3.3: Test Plan Export

As a user, I want to export the generated test plan, **so that** I can share it with my team.

Acceptance Criteria:

- ☐ Export as PDF
- ☐ Export as Word document
- ☐ Export as Markdown
- ☐ Preserve formatting
- ☐ Include JIRA reference

Tasks:

- TASK-3.3.1: Implement PDF export
- TASK-3.3.2: Implement Word export
- TASK-3.3.3: Implement Markdown export

- TASK-3.3.4: Add export buttons to UI
 - TASK-3.3.5: Test export quality
-

EPIC 4: Web Application Frontend

Epic Summary: Build user-friendly web interface for the Test Plan Agent

Story 4.1: Application Setup & Navigation

As a user, I want to access a clean web interface, **so that** I can easily navigate the application.

Acceptance Criteria:

- ☐ Modern, responsive UI design
- ☐ Sidebar navigation
- ☐ Dashboard/home page
- ☐ Settings page
- ☐ Test Plan Generator page

Tasks:

- TASK-4.1.1: Set up React/Vue frontend project
- TASK-4.1.2: Create layout component with sidebar
- TASK-4.1.3: Build navigation menu
- TASK-4.1.4: Create dashboard page
- TASK-4.1.5: Implement routing

Story 4.2: Settings Page

As a user, I want to configure all settings in one place, **so that** I can manage JIRA and LLM configurations.

Acceptance Criteria:

- ☐ JIRA connection settings section
- ☐ Groq settings section
- ☐ Ollama settings section
- ☐ Save/Reset buttons
- ☐ Validation messages

Tasks:

- TASK-4.2.1: Create settings page layout
- TASK-4.2.2: Add JIRA settings form
- TASK-4.2.3: Add Groq settings form

- TASK-4.2.4: Add Ollama settings form
- TASK-4.2.5: Implement save/reset functionality

Story 4.3: Test Plan Generator Page

As a user, I want to have a dedicated page for generating test plans, **so that** I can easily create test plans from JIRA tickets.

Acceptance Criteria:

- ☐ JIRA ID input field
- ☐ Fetch JIRA button
- ☐ Display JIRA details
- ☐ Template selection
- ☐ Generate Test Plan button
- ☐ Display generated test plan
- ☐ Export options

Tasks:

- TASK-4.3.1: Create generator page layout
 - TASK-4.3.2: Add JIRA input section
 - TASK-4.3.3: Add JIRA details display
 - TASK-4.3.4: Add template selector
 - TASK-4.3.5: Add generate button with loading state
 - TASK-4.3.6: Add test plan output display
 - TASK-4.3.7: Add export buttons
-

EPIC 5: Local & Cloud Deployment

Epic Summary: Support both local and cloud deployment options

Story 5.1: Local Deployment

As a user, I want to run the application locally, **so that** I can use it without internet connectivity.

Acceptance Criteria:

- ☐ Run backend locally (Python/Node.js)
- ☐ Frontend served locally
- ☐ Ollama integration works offline
- ☐ Local storage for settings
- ☐ Setup documentation

Tasks:

- TASK-5.1.1: Create local backend server
- TASK-5.1.2: Configure local frontend build
- TASK-5.1.3: Test Ollama offline functionality
- TASK-5.1.4: Create local setup script
- TASK-5.1.5: Write local deployment README

Story 5.2: Cloud Deployment Support

As a user, I want to deploy the application to cloud, so that my team can access it remotely.

Acceptance Criteria:

- ☐ Docker containerization
- ☐ Environment variable configuration
- ☐ Cloud platform support (AWS/Azure/GCP)
- ☐ Scalability considerations
- ☐ Cloud deployment documentation

Tasks:

- TASK-5.2.1: Create Dockerfile
- TASK-5.2.2: Create docker-compose.yml
- TASK-5.2.3: Add environment variable config
- TASK-5.2.4: Test cloud deployment
- TASK-5.2.5: Write cloud deployment README

Quick Reference: Issue Types

Issue Type	Description	Example
Epic	Large body of work	EPIC-1: JIRA Integration
Story	User-facing feature	STORY-1.1: JIRA Connection Setup
Task	Technical implementation	TASK-1.1.1: Create JIRA settings UI
Bug	Defect to fix	BUG-001: Connection timeout error

Sprint Planning Suggestion

Sprint 1: Foundation

- EPIC 1 (JIRA Integration)
- Basic frontend setup

Sprint 2: AI Integration

- EPIC 2 (LLM Integration)
- Settings page

Sprint 3: Core Feature

- EPIC 3 (Test Plan Generation)
- Generator page

Sprint 4: Polish & Deploy

- EPIC 4 (UI/UX improvements)
- EPIC 5 (Deployment)

Labels for Categorization

- jira-integration
- llm-integration
- frontend
- backend
- groq
- ollama
- test-plan
- high-priority
- documentation